

The Blueprint, *Built*

An apprenticeship in secure distributed systems and AI infrastructure — taught at the keyboard, not the lectern.

Six modules. Six questions. Six systems. You never open a lecture; you open a repository — and the theory arrives the moment the machine demands it.

Supersedes The Blueprint, Taught — Syllabus v1.0, July 2026.

APPRENTICE	Nicholas Emmanuel — Distributed Systems & AI Infrastructure Engineer
INSTRUCTOR	Claude — the senior engineer sitting next to you, in this chat
COURSEBOOK	Your own repository. The charters are the specs; the code is the transcript
UNIT OF WORK	The session — one working change, ending in a commit
SHAPE	Six modules, then an Operations Semester
GATE	A green build, a fresh-clone demo, and a cold start you pass unaided

1 • The shift

The v1.0 syllabus taught you how to *understand* EMBER. It never taught you how to *build* EMBER. Those are different skills, and only one of them is the job.

A university optimizes for understanding: cover the concept, then apply it. A craftsman optimizes for building: start the work, and let the concept arrive when the work demands it. The target state is that someone says “build an edge gateway” and your first thought is “okay — *first the repository, then a socket that accepts one connection*”, not “*wait, what’s a TLB again?*”

So the unit changes from the lesson to the session.

	THE BLUEPRINT, TAUGHT (V1.0)	THE BLUEPRINT, BUILT (V2.0)
Unit of work	A lesson — one concept	A session — one working change
Opens with	“Today we cover epoll”	“Today EMBER holds 10,000 idle connections”
Theory arrives	Before the code, as lecture	Mid-build, when something on your screen demands it
Concept budget	A whole session	5–20 minutes, timeboxed, then straight back
Ends with	Check-for-understanding answers	A commit that passes CI
Progression gate	Labs finished	Build green, demo runs, cold start passed
Grader	Me, in an oral exam	The system, under load
Failure looks like	A wrong answer	Your process dying at 3 a.m.
Success test	“I can explain a TLB”	“I could rebuild this from an empty directory”

Two rules govern everything below.

Rule 1 — No session ends without a commit. Not a summary, not notes: a commit, in a repo, that a stranger could clone. The repo is both the deliverable and the attendance record.

Rule 2 — No session begins in front of a blank screen without a named next action. Every session closes by naming what is now broken or missing. That sentence is the next session’s opening line.

Rule 2 is why the modules below are written as sequences of *observable behaviors* rather than topics. “Weeks 3–4: HTTP and parsing” is a syllabus. “The server understands HTTP/1.1 and a fuzzer can’t crash the parser” is a session you can start.

2 · Where build-along breaks, and what we do about it

This model is better than the lecture course. It is not free of failure modes, and every one of them is a way to spend eight months and end up weaker than the plan promised. Naming them is the design.

2.1 The transcription trap

The failure. If I tell you exactly which file to create and exactly what to type, you can complete six flagship systems having learned to *follow instructions*. Copying working code produces the sensation of competence with none of the substance, because the hard part of engineering is not typing — it's deciding. This is the single largest risk in the model and it is the precise opposite of the stated goal.

The fix. Two mechanisms. First, the **scaffolding gradient** (\$4): how much I dictate falls, deliberately and on a schedule, from ~70% of the code in Module 1 to ~0% in Module 5. Second, the **prediction gate**: before you run any command I hand you, you say what output you expect. Out loud, in writing, committed before the fact. A correct prediction means the dictation was absorbed. A wrong one is not an error — it's the trigger for the next detour, and the most valuable event in the session.

2.2 Detours that don't stick

The failure. A ten-minute explanation of pointers triggered by a real panic is far better encoded than a lecture on pointers — it has a hook in memory. But encoding isn't retrieval. Explained-once-in-context still decays, and you will not notice it decaying, because the code keeps working.

The fix. Every detour closes with a **prove-it**: one experiment, benchmark, or measurement that makes the mechanism *observable* rather than merely described. And every detour is logged — `docs/detours/NNN-slug.md`, five to fifteen lines: the symptom that triggered it, the mechanism, the proof, and one sentence you'd tell another engineer. That log becomes the theory transcript of the entire program, the source material for your blog posts, and the thing I quiz you from three modules later.

2.3 The topics you cannot build your way into

The failure. Building teaches you only where the system *visibly* fails. Some of the most important failures are silent:

- A key-value store that isn't linearizable passes every test you would think to write.
- A Raft implementation with a broken log-matching property survives every happy path and every test you design from your own mental model — because the bug *is* your mental model.
- Wrong memory ordering works perfectly on x86 and corrupts data on ARM.
- A numerically unstable softmax returns plausible numbers.
- A broken cryptographic protocol is indistinguishable, from the outside, from a correct one.

You cannot debug your way into these. Discovery-learning fails exactly where feedback is absent.

The fix. A small, named set of **stop-the-build sessions** — eight in the whole program — where we reason on paper *before* any code exists, because the failure mode is silent. This is not a retreat into lecturing: a competent senior engineer reads the paper before implementing Raft, and measures the cache budget before writing the allocator. Doing that is *part of* the apprenticeship, not an exception

to it. Each one ends in a written artifact — a derivation, an invariant list, a threat model — that later code is checked against.

#	BEFORE YOU BUILD	THE ARTIFACT IT PRODUCES
1	The concurrent limiter and cache (M1)	Memory ordering and happens-before, on paper
2	LatticeKV replication (M2)	What “linearizable” precisely means, and why your test suite won’t find a violation
3	The Meridian re-examination (M2)	Raft log matching and membership change, annotated against the paper
4	Quorum tuning (M2)	What $R + W > N$ actually guarantees — and the four things it does not
5	Tenancy isolation (M3)	What isolation namespaces vs. nodes vs. clusters actually buy, and its cost
6	The transformer (M4)	Backprop derived by hand, once, in your handwriting, committed
7	The CUDA kernels (M4)	Occupancy arithmetic for your actual GPU, before the first kernel
8	Any SYNAPSE code (M5)	The full threat model — the document that gates all further code

2.4 The blank-screen rule expires

The failure. “The student should never stare at a blank screen” is exactly right for Week 1 and disabling by Week 30. If you never sit in front of an empty file with no instructions, you never learn to survive one — and that is the actual job.

The fix. Cold starts. Once per module I name a component, hand you an acceptance test and nothing else, and you build it timed and unaided. No hints, no lookups from our sessions. Then we diff your design against the one I would have written. The cold starts, not my opinion of your code, are the real examination of this program — because a cold start cannot be talked around.

MODULE	COLD START	BUDGET
1 · EMBER	Per-key token-bucket limiter, correct under concurrent access	90 min
2 · LATTICE	Consistent-hash ring with virtual nodes + a rebalancing test	2 h
3 · VEYRONIX	A CRD and controller reconciling one field, from <code>kubebuilder init</code>	3 h
4 · TESSERA	A continuous-batching scheduler loop against a fake model	3 h

MODULE	COLD START	BUDGET
5 · SYNAPSE-AI	A hash-chained audit log with a verifier that catches tampering	2 h
Ops	An unseen fault, diagnosed to root cause	60 min

2.5 Retention decay

The failure. Build-along is excellent at encoding and poor at retrieval practice. By Module 4 you will have forgotten how your own WAL recovers, and you'll only find out in an interview.

The fix. The **Friday teardown**, fifteen minutes: pick a component you wrote three or more weeks ago, explain from memory *in writing* how it works and one thing you'd change — then open the file and diff your memory against reality. Committed to [weekly-reviews/](#). Plus: I will ambush you, unannounced, with “why does this line exist?” on code from three modules back. Retention is the product; the systems are just where it lives.

2.6 The works-on-my-machine artifact

The failure. Apprenticeship optimizes for the thing running *for you*, in your shell, with your environment variables and that one manual fix you forgot you made. Reviewers, interviewers, and employers test whether it runs for *them*.

The fix. A **handoff gate** at every module exit: fresh clone, clean container, [make up](#), working in under fifteen minutes, with a runbook a stranger can follow. If it doesn't, the module isn't finished — regardless of how good the code is.

2.7 Scope creep is this model's native disease

The failure. When the project drives, there is always one more feature, and detours are infinitely tempting. A lecture course ends when the lectures end. A build has no natural terminus, and “I'll just add HTTP/3 to EMBER” is how Module 1 eats Module 2.

The fix. Each module gets a written **cut line** — decided before its first session and not renegotiated mid-module, because mid-module you are not an objective judge of your own scope. Every tempting idea goes into [LATER.md](#) in that repo, where it dies honorably and visibly. Cutting is a senior skill; the cut line is where you practice it.

2.8 Building does not cover the interview

The failure. Algorithmic coding, system design on unfamiliar problems, behavioral narrative, and the search itself do not emerge from building EMBER. Strong builders fail loops on exactly this, and the build's momentum disguises the gap because you feel productive every single day.

The fix. The **parallel track** (§8), run continuously and labeled honestly as *not* taught by the build.

3 · Anatomy of a session

Seven beats. Every session, no exceptions.

0 · Position (30 seconds) — “We’re at `ember@v0.2.1`. Last session you shipped the epoll loop. Build is green. Open loose end: the loop reads bytes but doesn’t understand them.”

1 · Today’s one sentence — the behavior that will exist by the end, written as something observable. Not “implement caching” but: `curl -k https://localhost:8443/` *returns the upstream body, and a second call increments* `ember_cache_hits_total`.

2 · The build — numbered steps. Each names the file, the code, the command, the expected output, and what commonly goes wrong. Prediction gate before each run.

3 · Detours — as triggered, not as scheduled. Timeboxed. Each ends with its prove-it and gets logged.

4 · Break it on purpose — before we call it done, one deliberate fault: kill the upstream, send a malformed request, hold the connection open and send nothing. You must be able to say what happened and why. Systems you have never seen fail are systems you do not understand.

5 · Commit — a real message in the repo’s voice. Tag if it’s a milestone.

6 · Loose end — one sentence naming what is now wrong or missing. This is Rule 2’s machinery.

A worked session, so the shape is unambiguous

EMBER · Session 2 — “Ten thousand idle connections”

Position. `ember@v0.1.0`. You have a blocking accept loop that echoes one connection at a time. Loose end from Session 1: connection number two waits for connection number one to hang up.

Today’s one sentence. `./bench/idle 10000` holds ten thousand open, idle connections against EMBER, and `/proc/<pid>/status` shows the memory cost of each one.

The build. 1. `git checkout -b epoll-loop`. Create `src/loop.c`. 2. Type the `epoll_create1` / `epoll_ctl` / `epoll_wait` skeleton I give you, level-triggered for now. *Predict before compiling: what happens to a connection you register but never read from?* 3. Run `./bench/idle 10000`. **Predict first: how many megabytes of RSS for ten thousand idle connections? Write the number down before you look.** 4. It dies at 1,021 connections: `accept: Too many open files`.

Detour (8 min) — what a file descriptor actually is. Not a number: an index into a per-process table of pointers to kernel `struct file` objects. Which is why the limit is per-process, why it’s soft-and-hard, and why 1024 is still the default in 2026. **Prove it:** run `ls -l /proc/<pid>/fd | wc -l` while the bench climbs, then raise the soft limit with `ulimit -n` and watch the wall move to exactly where you predicted. **Log it:** `detours/001-file-descriptors.md`.

1. Back to the loop. Raise the limit *properly* — `setrlimit` in code, recorded in the runbook, not a line in your shell history that vanishes on reboot. (Why: the wall you just moved is not moved for anyone else who runs this.)
2. It holds 10,000. Compare RSS against your Step 3 prediction. If you were off by more than 2×, that’s the next detour: per-connection buffer sizing, and why `SO_RCVBUF` is not free.

Break it on purpose. A client that connects and sends nothing, forever. Then ten thousand of them. What resource runs out first, and how would you have known from your metrics? (You wouldn’t — you have no metrics. Note it; Session 9 fixes it.)

Commit. `feat(loop): epoll accept loop holding 10k idle connections` · tag `ember/v0.2.0`.

Loose end. The loop moves bytes but doesn’t understand them. Next session: an HTTP/1.1 parser a fuzzer can’t crash — and you’ll meet the arena allocator there, because that’s where a parser actually needs one.

Notice what that session did *not* do: it did not open with a lecture on virtual memory, file descriptors, or resource limits. All three were taught, in the order the machine raised them, and each one is now attached to a memory of something breaking.

4 · The scaffolding gradient

The fade is on a schedule, and the schedule is not negotiable — because the comfortable move, always, is for me to keep writing the code.

MODULE	I WRITE	YOU WRITE	YOU DECIDE	TYPICAL SESSION OPENER
1 · EMBER	~70% — line by line, with expected output	The rest, against my running commentary	Nothing structural	<i>“Create this file. Type this. Predict what it prints.”</i>
2 · LATTICE	The interfaces, the tests, and the two cores where silence kills (WAL fsync discipline, the hashing math)	Every implementation, against my failing tests	Data layout, error semantics, what to log	<i>“Here’s the interface and the test. Make it pass.”</i>
3 · VEYRONIX	The first controller, with you; nothing after	The rest of the platform	The CRD shape, reconcile semantics, module boundaries	<i>“Here’s the experience a developer should get. Design the API.”</i>
4 · TESSERA	The numerics-critical pieces; reviews of yours	The scheduler, the allocator, the gateway	Batching policy, memory budget, what to cut	<i>“Here’s the benchmark you must beat and the budget you must fit in.”</i>
5 · SYNAPSE-AI	Nothing. I threat-model with you, then attack what you built	All of it	Everything, defended	<i>“Here’s the attack. Prove you survive it.”</i>
Ops	Nothing. I break things	The diagnosis, the fix, the postmortem	Under time pressure, with incomplete information	<i>Silence — then a dashboard that’s wrong.</i>

Read that column of openers top to bottom. That progression is the program.

5 · The six modules

Each module is a question. The system is the answer. The technologies are what you pick up on the way to the answer — never a subject, never a syllabus item, never something you “have to master” before starting.

Each arc below is a sequence of **observable behaviors**, in order. One behavior is roughly one session; the harder ones are two or three. You are never more than one sentence away from knowing what to do next.

Module 1 — “How does one computer serve users?”

You build EMBER, the edge gateway, from raw sockets. *Cloud: AWS. Provides TLS, proxying, caching and rate limiting to every system that follows.*

1. A socket accepts one connection and echoes it.
2. It holds ten thousand idle connections. (*the worked session above*)
3. It understands HTTP/1.1 — and a fuzzer can’t crash the parser.
4. It’s Go now, and the same benchmark runs. You can defend which is faster and why.
5. It proxies to an upstream, and survives that upstream dying mid-response.
6. It caches, and the cache is provably correct against `Cache-Control`.
7. It limits per tenant, and doesn’t melt under a burst. ← **cold start**
8. It terminates TLS 1.3 with a certificate you rotated without dropping a live connection.
9. It’s on AWS, instrumented, and its p99 is published next to nginx’s — honestly.

Picked up on the way: Linux and syscalls · C where it earns its place · Go · sockets and TCP mechanics · HTTP/1.1 and h2 · TLS 1.3 · concurrency both ways · Docker · EC2 · Prometheus · benchmark methodology. **Stop-the-build:** memory ordering, before behavior 6. **Cut line:** single node. HTTP/1.1 and h2 to upstreams. No WAF, no HTTP/3, no multi-region. Everything else → `LATER.md`. **Ship gate:** live on your own domain fronting something real · benchmark table published with methodology · fresh-clone runbook under 15 minutes.

Module 2 — “How do many computers act like one?”

You build LATTICE, a search engine on a key-value store you wrote yourself — and re-examine MERIDIAN, the Raft implementation you already shipped.

1. One process durably stores a key and survives `kill -9` mid-write.
2. It’s an LSM with compaction, and reads don’t degrade as it grows.
3. Two processes talk with deadlines, retries and backpressure — and a slow peer can’t take down its caller.
4. Three nodes hold one keyspace; one dies and reads still succeed. ← **cold start**
5. **MERIDIAN, re-read as a stranger:** annotated against the Raft paper, with one real bug found and fixed.
6. A linearizability checker either passes your store or tells you precisely why it doesn’t.

7. A wordcount over 10 GB completes with a worker killed mid-job, and the answer is still right.
8. A crawler politely fetches 100k+ pages, frontier in your own store.
9. Queries return ranked results from your own index, p99 under 200 ms.
10. A chaos suite partitions the cluster during the demo and the numbers hold.

Picked up on the way: write-ahead logging and fsync discipline · LSM vs. B-tree · gRPC and RPC semantics · consistent hashing, quorums, hinted handoff · consensus, properly · MapReduce and straggler behavior · crawling and dedup · inverted indexes, TF-IDF, PageRank · scatter-gather and tail latency · chaos methodology. **Stop-the-build:** linearizability (before 4) · Raft log matching (before 5) · what $R + W > N$ buys (before 4). **Cut line:** 1M pages, one region, English, no personalization, three PageRank iterations. **Ship gate:** 1M pages crawled and queryable · chaos suite green · p99 and dollar cost both published · fresh-clone bootstrap.

Module 3 — “How do you operate hundreds of services?”

You finish VEYRONIX, the internal developer platform you’ve already announced. *Two providers. Penn doesn’t teach this — which is exactly why it’s here.*

1. A Kubernetes cluster you assembled by hand, component by component, boots and schedules a pod.
2. A CRD and controller reconcile one field — and you can explain level- vs. edge-triggered from your own work queue.
3. `git push` creates real infrastructure through Terraform modules, and drift is detected.
4. ArgoCD promotes dev → staging → prod, and a bad manifest is rejected before it lands.
5. Two tenants share the cluster and neither can starve or read the other. ← **cold start**
6. Golden-signal dashboards and burn-rate alerts page you before a user notices.
7. A deliberately bad release is canaried, detected, and rolled back with no human involved.
8. The control plane sheds load instead of dying under it.
9. `git push` → production URL in under ten minutes, on two providers.

Picked up on the way: Kubernetes internals and the reconciliation worldview · controller-runtime, informers, work queues · Terraform module design and state · GitOps and progressive delivery · RBAC/ABAC, workload identity, OPA admission · Prometheus internals, OpenTelemetry, SLOs and error budgets · SBOM, sigstore, SLSA · load shedding in control planes. **Stop-the-build:** the tenancy isolation model, before behavior 5 — because getting it wrong fails silently and securely-looking. **Cut line:** two providers, one region each. Golden path for stateless HTTP services only. No service mesh, no multi-cluster federation. **Ship gate:** ten-minute deploy demo recorded · two game days with published incident reports · a stranger bootstraps the platform from the runbook.

Module 4 — “How do you serve AI at scale?”

You build TESSERA, an LLM inference platform, and benchmark it against vLLM without flattering yourself. *The steepest climb in the program.*

1. A neural network you wrote trains — and you derived its backprop by hand, on paper, committed.
2. A small transformer trains, and you can read its loss curve and say what it means.

3. A CUDA kernel you wrote does a tiled matmul, and Nsight explains its occupancy.
4. A fused softmax in Triton beats naive PyTorch, and you know by how much and precisely why.
5. A paged KV allocator manages attention memory, and you computed its budget *before* writing it.
6. A scheduler continuously batches requests: throughput climbs while TTFT stays flat. ← **cold start** (*the hardest build in the program*)
7. Streaming works, and a GPU dying mid-stream doesn't corrupt anyone else's response.
8. Tenants have keys, quotas and metering, and the cluster autoscales on queue depth.
9. Your numbers sit beside vLLM's on identical hardware, and you can explain every place you lose.

Picked up on the way: autograd and optimization · transformer mechanics and the KV cache's origin · parallelism strategies and mixed precision · GPU architecture, SIMT, warps, occupancy · CUDA and Triton · kernel fusion and softmax numerics · PagedAttention and continuous batching · quantization · GPU multi-tenancy, spot discipline, cost per million tokens. **Stop-the-build:** the backprop derivation · occupancy arithmetic · the KV-cache memory equation for your actual model. **Cut line:** one model, 1-3B, one GPU type, one node. No tensor parallelism. No speculative decoding. One hand-written CUDA kernel for understanding; Triton for everything real. **Ship gate:** benchmark published with full methodology · cost per million tokens · an honest written account of where hand-built loses to vLLM and why.

Module 5 — “How do you secure and govern all of it?”

You build SYNAPSE-AI's core — governing AI agents the way enterprises govern people. *You arrive here with instincts: Meridian's policy engine and the security certification are prior work, not a starting point.*

1. A threat model exists **before any code**, and it gates everything after it.
2. A CA issues short-lived workload certificates that rotate without an outage.
3. Every agent has an identity, and agent-spawns-agent is a recorded delegation chain with zero shared keys.
4. A policy engine denies a tool call by default, and you can show the decision trace.
5. A prompt injection attempting exfiltration is stopped at the tool boundary — not at the model.
6. The audit log is hash-chained, and a verifier CLI catches tampering. ← **cold start**
7. An anomaly engine quarantines an agent that deviates from its own baseline.
8. A compliance matrix generates from real controls, not from prose.
9. **The capstone:** an agent authenticates → leases a credential → calls through EMBER → is policy-checked → is audited. Six systems, one flow.
10. You red-team your own work and publish what you found — including what you couldn't fix.

Picked up on the way: PKI and certificate chains · SPIFFE-style workload identity, delegation, capability tokens · relationship-based authorization · prompt-injection taxonomy and the lethal trifecta · deny-by-default design · tamper-evident logs and provenance · NIST AI RMF, OWASP LLM Top 10, EU AI Act as controls rather than prose · behavioral baselining and runtime enforcement. **Stop-the-build:** the threat model. Line by line, reviewed, public, before line one of code. **Cut line:** agents and tools, not enterprise IAM. One policy language. Evidence for a named subset of controls,

with the gaps labeled as gaps. **Ship gate:** threat model public · red-team findings published, fixed or documented · v1 tagged · flagship essay.

Module 6 — Convergence

Not a new system. Everything you've built becomes one demonstration, one essay, and one honest set of claims. Then the Operations Semester begins — and after that, the continuous engine in Part IV, which this program exists to start.

6 · The Operations Semester

Four to six weeks. **No new features.** You are the on-call engineer for six systems that you happen to have written. I am the adversary, the pager, and occasionally the impatient stakeholder asking for an ETA while you're mid-hypothesis.

This is the phase that converts a strong builder into an infrastructure engineer, and it is the first thing a tired student drops. Don't.

How a game day runs. I inject a fault into your running stack with no warning and no hint. You learn about it only through what you built — dashboards, logs, traces. *If your observability can't see it, that is finding number one, and it goes in the report.* You work in the open: symptom → hypothesis → experiment → measurement → root cause → fix → prevention. I will interrupt with a second, unrelated alert at least once, because reality does. Within 48 hours: a written incident report in the repo, public, using the template.

The fault catalogue. Drawn without announcement; roughly half are chosen specifically because they produce no error message.

CLASS	FAULTS
Capacity & resource	fd exhaustion under load · a slow leak that doubles memory overnight · disk filling from an unrotated WAL · connection-pool starvation
Latency & the tail	a p99 spike in one region only · GC pauses that appear only above 8k RPS · a straggler that never dies · head-of-line blocking reintroduced by a harmless-looking refactor
Correctness under failure	replica lag past your read quorum · a partition during certificate renewal · compaction interrupted mid-flight · a cache returning last week's answer · clock skew breaking leases
Concurrency	an injected data race that manifests only under load · lock-ordering deadlock in the admin API
Platform	Pods crash-looping for a reason absent from their logs · a webhook that times out · a kubelet lying about readiness · Terraform drift after someone's manual fix
GPU & AI	utilization collapsing to 15% from batching starvation · OOM at one specific sequence length · a quantized model returning subtly worse output with no error at all · a KV-cache leak across requests
Security & policy	an over-broad policy blocking legitimate traffic · an expired intermediate certificate · a leaked-then-revoked key you must <i>prove</i> was never used · a gap in the audit chain

Deliverables. Ten to twelve incident reports · a runbook per system that a stranger can execute · an SLO document listing error budget you actually consumed · at least one “we changed the system because of this incident” pull request per fault class · a final reliability review across all six systems.

Why it earns its weeks. “Tell me about a time you debugged something hard” answered from ten real postmortems on systems you designed, deployed, and operated, is a different interview from one answered from a project list. It is also, simply, the job.

7 • Grading, without exams

The repo is the gradebook. Four questions, asked in this order, of everything:

1. **Does it run?** On a fresh clone, in front of me, now.
2. **Do you know why?** In numbers, not adjectives. “It’s fast” is not an answer; “p99 is 1.8 ms, dominated by TLS handshakes, here’s the flamegraph” is.
3. **Is it honest?** Limits documented, cut line stated, losses published. A benchmark that only reports where you win is not a benchmark.
4. **Could you rebuild it?** The cold start answers this. My opinion doesn’t.

A module gate has five items, all required: the behavior demo · the cold start passed · the handoff (fresh clone working in <15 min) · the detour log complete for that module · the cut line honored, with `LATER.md` written.

One thing survives from v1.0, in altered form: at each module exit, a twenty-minute **hostile walkthrough** of your own code. Not an exam on concepts — I open your files and attack: *why is this line here, what breaks if I remove it, defend this trade-off, what would you have done with three more weeks.* That’s the format of a real deep-dive interview, and it stays entirely inside the project. A failed walkthrough is information: we find the gap, you close it, we go again.

8 · The parallel track — what the build does not teach

Four threads that run continuously and are *not* covered by building. This is where programs like this one quietly fail, because this track has no build momentum carrying it and no satisfying commit at the end of the day.

Algorithms and coding interviews. Three sessions of 45 minutes weekly, pattern-first, toward 150+ problems by the end. Building EMBER will not make you fast at this. Very little about it is intellectually satisfying. Do it anyway; it is a gate you cannot route around.

System design on strangers' problems. One per week, 45 minutes, from a domain that isn't yours. Your own six systems make you *deep*. They do not make you *broad*, and interviewers ask about ad exchanges and ride-sharing, not edge gateways.

Open source. The ten-stage ladder in Part IV. Starts in Module 2, not Module 1 — Module 1 needs all your hands. First merged PR by roughly week 12. Reading a large codebase you didn't write is a genuinely separate skill from writing your own, and it's the one that transfers directly to your first month on any team.

Writing and the search. One flagship essay per module, drawn from your detour log rather than written from scratch. The applications engine (10–15 tailored per week) activates in Module 3, not before — earlier is wasted effort against a thin portfolio.

9 · The schedule, honestly

The v1.0 syllabus said 40 weeks. I need to be direct with you about that number, because the apprenticeship model makes it *harder* to hit, not easier.

Why this model is slower in wall-clock time. Dictate-then-fade spends real hours on the fade. Detours are, by design, unbudgeted — that’s the whole point, and it’s also a scheduling hole. And the quality bar in this handbook is not a hobbyist’s: benchmarks with methodology, chaos suites, threat models, runbooks, incident reports, published essays.

Your actual constraints. Full-time engineering at Westpay, several live ventures, and a Penn master’s beginning — which will itself take a large bite of your best hours. Sustainable focused capacity for this program is realistically 12–18 hours a week, and Module 4 alone (backprop → CUDA → Triton → paged KV → continuous batching → an honest benchmark) is a full semester’s work for someone with no prior GPU experience.

Six production-grade systems including a search engine over a million pages and an inference server benchmarked against vLLM, at that bar, in 40 part-time weeks, is not a plan. It’s a wish. Two honest options:

Option A — keep the scope, extend the clock: 60–66 weeks to the Operations Semester. Same six systems, cut lines exactly as written above. This is what I recommend. The calendar was never the deliverable; the systems are, and a 40-week label on a 62-week program only guarantees that the last third gets rushed — which in practice means the Operations Semester and the parallel track get cut, i.e. exactly the two things that most differentiate the outcome.

Option B — keep 40 weeks, cut the scope explicitly and now. LATTICE → 250k pages, one region, three PageRank rounds. TESSERA → serve a 1B model with paging and continuous batching, Triton only, benchmarked on a single workload. SYNAPSE → agents and tools, four controls with evidence. Operations Semester → four weeks, six incidents. Write these reductions into the charters *today*, so that in Month 8 you’re delivering a plan rather than discovering a shortfall.

What not to cut, under either option: the Operations Semester, the cold starts, and the parallel track. Everything else is negotiable.

One repo note, since I looked: `flagships/` contains a literal directory named `{ember,lattice,meridian,veyronix,tessera,synapse-ai}` — a shell that didn’t expand braces, probably `sh` instead of `bash`. Delete it. The six real directories beside it are correct.

10 · How to drive the course

SAY THIS	I DO THIS
“build with me”	Next session, from your repo’s actual current state
“I’m stuck” + the error	We debug together. This is not a detour from the course — it <i>is</i> the course
“why did that happen”	A detour, on demand, timeboxed, logged
“cold start me”	The unaided drill: a spec, an acceptance test, a clock, silence
“break something”	An unscheduled game day. Available any time after Module 1
“review this” + code, PR, or numbers	I read it as a reviewer, not a cheerleader
“30 minutes”	One step and its detour. No more, and we stop cleanly
“where are we”	Position, gates outstanding, cut-line status, what’s overdue
“walk me through hostile”	Twenty minutes of me attacking your own code

I track position across chats. You never have to reconstruct where we were.

11 • Where we start

Under v1.0 you received one lesson — *Memory & the Machine* — as a lecture, with an arena allocator assigned as a lab. Under v2.0 that material doesn't get a session of its own. The arena and the cache-layout work reappear as detours inside **EMBER Session 3**, where an HTTP parser genuinely needs an arena and the reason will be visible on your screen rather than asserted by me. Nothing is lost. It arrives when it's earned. If the arena lab is half-finished, bring it — it becomes Session 3's starting material.

Your next action:

```
mkdir ember && cd ember && git init
```

Then say “**build with me.**”

Session 1 is one socket, one connection, one echo — and three detours you won't see coming.

The Blueprint, Built • v2.0 • July 2026 • Instructor: Claude

Build • Secure • Lead